

M365 Speicher-Dashboard

Setup-Anleitung Microsoft 365 / Entra ID · Stand 14.08.2026

1 Übersicht

Das Dashboard liest die Speicherbelegung aller Postfächer, OneDrives und SharePoint-Websites über die Microsoft-Graph-Reports-API aus. Dafür braucht es eine **App-Registrierung** im Entra-ID-Tenant des Kunden. Am Ende dieser Anleitung liegen drei Werte vor, die im Dashboard unter **Einstellungen** (Zahnrad oben rechts) eingetragen werden:

Wert	Entspricht in Entra ID	Beispiel
Tenant-ID	Verzeichnis-ID (Mandant)	f6a2...-...-...9c1b (GUID)
Client-ID	Anwendungs-ID der App-Registrierung	3d84...-...-...77e2 (GUID)
Client-Secret	Geheimer Clientschlüssel (Wert, nicht ID!)	nur einmal sichtbar

Benötigte Graph-Berechtigungen (Typ: **Anwendung**, mit Administratorzustimmung):

Berechtigung	Zweck	Pflicht?
Reports.Read.All	Usage-Reports: Postfach-, OneDrive- und SharePoint-Belegung	ja
User.Read.All	Lizenzzuweisungen pro Benutzer, Erkennung freigegebener Postfächer	empfohlen
Organization.Read.All	Lizenz-Stammdaten (SKU-Namen)	empfohlen

Ohne die beiden empfohlenen Berechtigungen läuft die Speicher-Erfassung trotzdem — das Dashboard zeigt dann einen Hinweis und lässt Lizenzspalte und Aufschlüsselung leer.

2 Voraussetzungen

- Konto mit der Rolle **Globaler Administrator** (für App-Registrierung, Admin-Consent und Berichtseinstellungen).
- Für den PowerShell-Weg (Abschnitte 4–6): **PowerShell 7** und das Microsoft-Graph-Modul: `Install-Module Microsoft.Graph.Applications, Microsoft.Graph.Reports -Scope CurrentUser`
- Die Einrichtung ist einmalig pro Tenant und dauert ca. 10 Minuten.

3 Weg A: Einrichtung im Portal (klickbar)

Wer lieber scriptet: Abschnitt 4 erledigt alle folgenden Schritte automatisch.

3.1 App registrieren

1. <https://entra.microsoft.com> öffnen und als Administrator anmelden.
2. Navigation: **Identität > Anwendungen > App-Registrierungen > Neue Registrierung**.
3. Name: z. B. **Speicher-Dashboard**. Kontotyp: **Nur Konten in diesem Organisationsverzeichnis** (einzelner Mandant). Umleitungs-URI leer lassen.

4. **Registrieren** klicken. Auf der Übersichtsseite erscheinen jetzt **Anwendungs-ID (Client)** und **Verzeichnis-ID (Mandant)** — beide notieren.

3.2 Berechtigungen erteilen

1. In der App-Registrierung: **API-Berechtigungen > Berechtigung hinzufügen > Microsoft Graph**.
2. **Anwendungsberechtigungen** wählen (nicht «Delegierte Berechtigungen»!).
3. Suchen und anhängen: **Reports.Read.All, User.Read.All, Organization.Read.All** — dann **Berechtigungen hinzufügen**.
4. Wichtig: **Administratorzustimmung für <Organisation> erteilen** klicken und bestätigen. Alle drei Zeilen müssen danach einen grünen Haken «Gewährt» zeigen.

3.3 Client-Secret erstellen

1. **Zertifikate & Geheimnisse > Neuer geheimer Clientschlüssel**.
2. Beschreibung z. B. **Dashboard**, Gültigkeit **24 Monate** (Maximum) — **Hinzufügen**.
3. Die Spalte **Wert** sofort kopieren (nicht die «Geheimnis-ID»!). Der Wert ist nur unmittelbar nach dem Erstellen sichtbar.
4. Ablaufdatum im Kalender vormerken: nach Ablauf schlägt die Erfassung fehl, bis ein neues Secret erstellt und im Dashboard hinterlegt wird.

4 Weg B: Einrichtung per PowerShell (automatisch)

Das Script `setup/01-register-app.ps1` aus dem Projektordner erledigt Registrierung, Berechtigungen, Admin-Consent und Secret in einem Lauf und gibt die drei Zielwerte aus. Beim Start öffnet sich eine Anmeldung — dort mit dem Administratorkonto des Kunden-Tenants anmelden.

```

# 01-register-app.ps1 – legt die App-Registrierung für das Speicher-Dashboard
# vollautomatisch an: App + Service Principal, Graph-Berechtigungen inkl.
# Admin-Consent, Client-Secret (24 Monate). Gibt am Ende die drei Werte aus,
# die im Dashboard unter Einstellungen eingetragen werden.
#
# Voraussetzungen:
#   Install-Module Microsoft.Graph.Applications -Scope CurrentUser
#   Ausführung als Global Administrator (oder Privileged Role Admin + App Admin)

$ErrorActionPreference = 'Stop'
$appName = 'Speicher-Dashboard'

Import-Module Microsoft.Graph.Applications
Connect-MgGraph -Scopes 'Application.ReadWrite.All', 'AppRoleAssignment.ReadWrite.All' -NoWelcome

# Microsoft-Graph-Anwendungsberechtigungen (App-Rollen, IDs sind global fix)
$graphAppId = '00000003-0000-0000-c000-000000000000'
$roles = [ordered]@{
    'Reports.Read.All' = '230c1aed-a721-4c5d-9cb4-a90514e508ef' # Pflicht: Usage-Reports
    'User.Read.All' = 'df021288-bdef-4463-88db-98f22de89214' # optional: Lizenzen
    'Organization.Read.All' = '498476ce-e0fe-48b0-b801-37ba7e2685c6' # optional: SKU-Namen
}

$existing = Get-MgApplication -Filter "displayName eq '$appName'"
if ($existing) {
    Write-Host "Es existiert bereits eine App-Registrierung '$appName':" -ForegroundColor Yellow
    Write-Host "  AppId $($existing.AppId)"
    Write-Host 'Abbruch – zum Auslesen der Werte 02-get-ids.ps1 verwenden.'
    return
}

Write-Host "Erstelle App-Registrierung '$appName' ..."
$access = @{
    resourceAppId = $graphAppId
    resourceAccess = @($roles.Values | ForEach-Object { @{ id = $_; type = 'Role' } })
}
$app = New-MgApplication -DisplayName $appName -SignInAudience 'AzureADMyOrg' `
    -RequiredResourceAccess @($access)

Write-Host 'Erstelle Service Principal ...'
$sp = New-MgServicePrincipal -AppId $app.AppId

Write-Host 'Erteile Admin-Consent für die Graph-Berechtigungen ...'
$graphSp = Get-MgServicePrincipal -Filter "appId eq '$graphAppId'"
foreach ($name in $roles.Keys) {
    New-MgServicePrincipalAppRoleAssignment -ServicePrincipalId $sp.Id `
        -PrincipalId $sp.Id -ResourceId $graphSp.Id -AppRoleId $roles[$name] | Out-Null
    Write-Host "  + $name"
}

Write-Host 'Erstelle Client-Secret (Laufzeit 24 Monate) ...'
$secret = Add-MgApplicationPassword -ApplicationId $app.Id -PasswordCredential @{
    displayName = 'Dashboard'
    endDateTime = (Get-Date).AddMonths(24)
}

$tenantId = (Get-MgContext).TenantId

Write-Host ''
Write-Host '=== Diese Werte im Dashboard unter Einstellungen eintragen ===' -ForegroundColor Green
[pscustomobject]@{
    'Tenant-ID' = $tenantId
    'Client-ID' = $app.AppId
    'Client-Secret' = $secret.SecretText
    'Secret-Ablauf' = $secret.EndDateTime.ToString('yyyy-MM-dd')
} | Format-List
Write-Host 'Das Secret ist nur jetzt sichtbar – sofort kopieren!' -ForegroundColor Yellow

```

Das Secret wird nur ein einziges Mal angezeigt. Direkt aus der Konsolenausgabe kopieren und im Dashboard eintragen. Die App-Rollen-GUIDs im Script sind von Microsoft global festgelegt und in jedem Tenant identisch (siehe Anhang).

5 IDs auslesen und Registrierung prüfen

Wenn die App-Registrierung bereits existiert (oder von jemand anderem angelegt wurde), liefert `setup/02-get-ids.ps1` Tenant-ID und Client-ID, listet die Secret-Laufzeiten und prüft, ob alle drei Berechtigungen mit Admin-Consent vorhanden sind:

```
# 02-get-ids.ps1 – liest die für das Dashboard nötigen IDs aus dem Tenant aus
# und prüft, ob die App-Registrierung vollständig ist (Berechtigungen, Consent,
# Secret-Ablauf). Ein neues Secret kann dieses Script nicht anzeigen – Secrets
# sind nur bei der Erstellung sichtbar (bei Bedarf 01-register-app.ps1 bzw.
# Portal: Zertifikate & Geheimnisse -> neues Secret).
#
# Voraussetzungen:
#   Install-Module Microsoft.Graph.Applications -Scope CurrentUser

$ErrorActionPreference = 'Stop'
$AppName = 'Speicher-Dashboard' # ggf. an den gewählten App-Namen anpassen

Import-Module Microsoft.Graph.Applications
Connect-MgGraph -Scopes 'Application.Read.All' -NoWelcome

$tenantId = (Get-MgContext).TenantId
Write-Host "Tenant-ID: $tenantId" -ForegroundColor Green

$app = Get-MgApplication -Filter "displayName eq '$AppName'"
if (-not $app) {
    Write-Host "Keine App-Registrierung '$AppName' gefunden." -ForegroundColor Yellow
    Write-Host 'Vorhandene App-Registrierungen:'
    Get-MgApplication -All | Select-Object DisplayName, AppId | Format-Table
    return
}
Write-Host "Client-ID: $($app.AppId)" -ForegroundColor Green

# Secret-Laufzeiten
if ($app.PasswordCredentials) {
    Write-Host ''
    Write-Host 'Client-Secrets:'
    $app.PasswordCredentials | ForEach-Object {
        $status = if ($_.EndDateTime -lt (Get-Date)) { 'ABGELAUFEN' }
        elseif ($_.EndDateTime -lt (Get-Date).AddDays(60)) { 'läuft bald ab' }
        else { 'gültig' }
        $until = $_.EndDateTime.ToString('yyyy-MM-dd')
        Write-Host (" {0} bis {1} [{2}]" -f $_.DisplayName, $until, $status)
    }
}
else {
    Write-Host 'Kein Client-Secret vorhanden –' -ForegroundColor Yellow
    Write-Host 'im Portal oder mit 01-register-app.ps1 erstellen.' -ForegroundColor Yellow
}

# Berechtigungen + Admin-Consent prüfen
$required = 'Reports.Read.All', 'User.Read.All', 'Organization.Read.All'
$graphSp = Get-MgServicePrincipal -Filter "appId eq '00000003-0000-0000-c000-000000000000'"
$sp = Get-MgServicePrincipal -Filter "appId eq '$($app.AppId)'"
$granted = @()
if ($sp) {
    $granted = Get-MgServicePrincipalAppRoleAssignment -ServicePrincipalId $sp.Id |
        ForEach-Object { ($graphSp.AppRoles | Where-Object Id -eq $_.AppRoleId).Value }
}
Write-Host ''
Write-Host 'Berechtigungen (Admin-Consent):'
foreach ($r in $required) {
    $ok = $granted -contains $r
    $mark = if ($ok) { 'OK ' } else { 'FEHLT' }
    $color = if ($ok) { 'Green' } else { 'Red' }
    Write-Host (" [{0}] {1}" -f $mark, $r) -ForegroundColor $color
}
if (-not $sp) {
    Write-Host 'Kein Service Principal –' -ForegroundColor Red
    Write-Host 'im Portal einmal Admin-Consent erteilen.' -ForegroundColor Red
}
```

Die Tenant-ID lässt sich zur Not auch ohne Anmeldung ermitteln — sie ist öffentlich pro Domain abrufbar:

```
# Tenant-ID über die Kundendomain ermitteln (ohne Anmeldung)
$domain = "kunde.ch"
$url = "https://login.microsoftonline.com/$domain/v2.0/.well-known/openid-configuration"
(Invoke-RestMethod $url).issuer
# -> https://login.microsoftonline.com/<TENANT-ID>/v2.0
```

6 Klarnamen in Berichten aktivieren (wichtig!)

Microsoft pseudonymisiert die Usage-Reports in vielen Tenants standardmässig: Statt Namen, UPNs und Site-URLs liefern die Reports dann nur Hashes. Das Dashboard erkennt das und zeigt eine rote Warnung. Die Einstellung muss einmalig abgeschaltet werden — im Admin Center oder per Script.

Im Admin Center:

1. <https://admin.microsoft.com> > **Einstellungen** > **Organisationseinstellungen** > Reiter **Dienste** > **Berichte**.
2. Haken bei «**In allen Berichten verborgene Benutzer-, Gruppen- und Websitenamen anzeigen**» so setzen, dass Klarnamen angezeigt werden (Pseudonymisierung aus) — **Speichern**.

Per PowerShell (setup/03-report-settings.ps1):

```
# 03-report-settings.ps1 – prüft die Pseudonymisierung der Microsoft-365-Berichte
# und schaltet sie auf Wunsch ab, damit das Dashboard Klarnamen und Site-URLs
# statt Hashes erhält. (Entspricht Admin Center: Einstellungen ->
# Organisationseinstellungen -> Berichte.)
#
# Voraussetzungen:
#   Install-Module Microsoft.Graph.Reports -Scope CurrentUser

$erroractionpreference = 'Stop'

Import-Module Microsoft.Graph.Reports
Connect-MgGraph -Scopes 'ReportSettings.ReadWrite.All' -NoWelcome

$settings = Get-MgAdminReportSetting
if ($settings.DisplayConcealedNames) {
    Write-Host 'Berichte sind aktuell PSEUDONYMISIERT (Hashes statt Namen).' -ForegroundColor Yellow
    $answer = Read-Host 'Pseudonymisierung jetzt abschalten? (j/n)'
    if ($answer -eq 'j') {
        Update-MgAdminReportSetting -BodyParameter @{ displayConcealedNames = $false }
        Write-Host 'Erledigt – Berichte zeigen künftig Klarnamen.' -ForegroundColor Green
        Write-Host 'Hinweis: Bereits erzeugte Reports ändern sich rückwirkend nicht;'
        Write-Host 'die nächste Erfassung (spätestens am Folgetag) enthält Klarnamen.'
    }
}
else {
    Write-Host 'Berichte zeigen bereits Klarnamen – nichts zu tun.' -ForegroundColor Green
}
```

Die Änderung wirkt erst auf **neu erzeugte** Reports. Da Microsoft die Reports mit ca. 24–48 h Verzögerung aktualisiert, kann es bis zum Folgetag dauern, bis die Erfassung Klarnamen liefert.

7 Werte im Dashboard eintragen und testen

1. Dashboard starten (pwsh ./start.ps1) und <http://localhost:8080> öffnen.
2. Oben rechts **Einstellungen** (Zahnrad) öffnen, Tenant-ID, Client-ID und Client-Secret eintragen. Falls der **Demo-Modus** aktiv ist: Haken entfernen.

3. **Verbindung testen** klicken — es muss «Verbindung erfolgreich — Token erhalten» erscheinen.
4. **Speichern**, dann **Jetzt erfassen**. Nach wenigen Sekunden füllen sich Kacheln, Diagramm und Tabellen.
5. Fertig — ab jetzt erfasst der eingebaute Wochen-Schedule (Montag 06:00) automatisch, solange der Server läuft.

Vorher im Demo-Modus getestet? Für einen sauberen Start ohne Demodaten den Server stoppen (pwsh `./stop.ps1`), die Datei `data/usage.db` löschen und neu starten.

8 Fehlerbehebung

Symptom	Ursache und Lösung
«Verbindung fehlgeschlagen ... AADSTS7000215»	Ungültiges Client-Secret — vermutlich wurde die «Geheimnis-ID» statt des «Werts» kopiert, oder das Secret ist abgelaufen. Neues Secret erstellen und den Wert eintragen.
«Verbindung fehlgeschlagen ... AADSTS700016»	Client-ID falsch oder App-Registrierung im falschen Tenant angelegt. IDs mit <code>02-get-ids.ps1</code> prüfen.
«Verbindung fehlgeschlagen ... AADSTS90002»	Tenant-ID falsch (Tippfehler oder GUID eines anderen Tenants).
Erfassung: Fehler 403 (Forbidden)	Admin-Consent fehlt oder Berechtigung wurde als «Delegiert» statt «Anwendung» angelegt. Abschnitt 3.2 wiederholen; Prüfung mit <code>02-get-ids.ps1</code> .
Rote Warnung «Benutzernamen sind pseudonymisiert»	Berichtseinstellung noch aktiv (Abschnitt 6). Nach dem Umstellen bis zum Folgetag warten und neu erfassen.
Banner «Lizenzinformationen fehlen»	User.Read.All / Organization.Read.All fehlen oder ohne Consent. Optional — Speicherdaten funktionieren trotzdem.
Erfassung leer / 0 Benutzer	Frisch eingerichteter Tenant oder Reports noch nicht generiert — Microsoft erzeugt die Reports mit bis zu 48 h Verzug.
Werte wirken 1–2 Tage alt	Normal: Die Reports-API liefert immer den Stand von vor ca. 24–48 h (Report Refresh Date).

Anhang: Feste Microsoft-GUIDs

Diese IDs sind von Microsoft global definiert und in jedem Tenant identisch — sie stehen so auch in den Scripts:

Objekt	GUID
Microsoft Graph (Resource-App)	00000003-0000-0000-c000-000000000000
App-Rolle Reports.Read.All	230c1aed-a721-4c5d-9cb4-a90514e508ef
App-Rolle User.Read.All	df021288-bdef-4463-88db-98f22de89214
App-Rolle Organization.Read.All	498476ce-e0fe-48b0-b801-37ba7e2685c6